Printmaker System Visual Map

Working architecture snapshot of the current platform. This map is meant to support planning, API hardening, and safer system changes without touching pricing or tenant isolation accidentally.

Based on `README.md`, `PROJECT_STATUS.md`, `docs/ARCHITECTURE_BOUNDARIES.md`, `docs/REST_API_READINESS_DRAFT.md`, `docs/SITE_DESIGN_V2_STATUS.md`, `docs/PRICING_SYSTEM.md`, `src/App.tsx`, and `src/pages/Admin.tsx`.

1. Platform Overview

One application currently carries storefront, admin, design, imports, and tenant management.

STOREFRONT RUNTIME

Customer-facing runtime

- Front page / shop
- Product pages
- Matrix pricing / product configuration
- Checkout-related flows
- Content / company pages

ADMIN RUNTIME

Operational control center

- Products and pricing systems
- Branding / Site Designer V1
- Branding / Site Designer V2
- Tenant / domain / publish control
- POD v1 and POD v2

DESIGNER MODULE

Production design toolchain

- Canvas / Fabric editor
- Preflight checks
- PDF export
- Color proofing

IMPORT LAYER

Supplier ingestion

- Wir-machen-druck imports
- Pixart imports
- Other supplier scripts

Supabase Auth | Supabase Database | Supabase Storage | Edge Functions

2. Runtime Flow

The publishing path is the critical line for branding, preview, and storefront behavior.

Admin UI
User edits settings, pages, colors, content, and product visibility.

→

Draft State
Temporary changes live here first and should be safe to test.

→

Preview Runtime
Preview reflects draft and published combinations.

→

Publish Action
Approved state is pushed forward as tenant configuration.

→

Tenant Published Config
Published tenant state becomes the runtime source.

→

Live Storefront
Customer sees final output and stable pricing/product behavior.

3. Domain Boundaries

These boundaries should become hard contracts, not just mental models.

STRONGEST CORE

Domains that should lead the architecture

- Pricing
- Product catalog basics
- Published price reads
- Tenant resolution basics

BOUNDARY TENSION

Areas where expectations and runtime can diverge

- Branding V2 vs actual storefront wiring
- Preview vs published behavior
- Master/tenant copy and distribution behavior
- UI configuration carrying business logic

STRICT ISOLATION

Must stay separate

- POD v1
- POD v2
- Designer logic must not own pricing

4. Stability Map

Not all domains are equally ready for API extraction or aggressive refactoring.

STABLE CORE

- Product catalog basics
- Matrix pricing
- Published price reads
- Tenant resolution basics

STABLE WITH CAUTION

- Storefront runtime
- Product detail rendering
- Reliable supplier import paths

BETA / INCOMPLETE

- Branding V2 runtime wiring
- Per-page theme controls
- Advanced preview / publish expectations

ISOLATED / SPECIAL HANDLING

- POD v1
- POD v2
- Any cross-tenant publish/sync logic

5. API Hardening Sequence

Expose stable behavior behind contracts. Do not export instability into APIs.

1. Stabilize
Fix module behavior in UI/runtime until it is reliable.

→

2. Declare Ready
Mark the module ready for API hardening.

→

3. Define Contract
Request/response, errors, tenant scoping, rollback.

→

4. Read-only API
Wrap existing logic first. No rewrite.

→

5. Shadow Compare
Old path and API path must match under real data.

→

6. Feature Flag
Roll out safely. Writes come later.

6. Target Contract-Based Architecture

The platform should be organized by contracts first, domains second.

- Tenant Context Contract
- Catalog Contract
- Product Configuration Contract
- Pricing Read Contract
- Branding / Theme Contract
- Publish / Visibility Contract
- Asset / Media Contract

Storefront Runtime	Product Admin	Site Designer	Designer Module
Tenant Distribution	Import Jobs	POD v1	POD v2

Practical rule: UI experiments should not redefine business rules. Contracts should outlive UI changes.